



aco-selection-parallel — Guia pro Time

O que foi criado

O repositório `aco-selection-parallel` agora tem **58 Issues** organizadas em 3 níveis:

Nível	Quantidade	O que representa
Épico	5	Milestones de alto nível — uma fase inteira do projeto
User Story	18	Entregáveis com valor — descreve O QUE fazer e POR QUÊ
Task	35	Unidade de trabalho atômica — descreve COMO fazer, qual arquivo, qual função

Os 5 Épicos (ordem de execução)

#	Épico	Prioridade
1	Versão Sequencial Refatorada — adaptar baseline em C++ puro, sem NumCpp	● Crítico
2	Paralelização com CUDA — kernels GPU (distância, formigas, feromônio, 1-NN)	● Crítico
3	Paralelização com OpenMP — pragmas de paralelismo CPU multi-thread	● Médio
4	Benchmark e Avaliação — pipeline de testes, prof ling Perf, gráficos	● Crítico
5	Documentação e Artigo — artigo acadêmico + apresentação final	● Médio

Labels (como filtrar)

Cada issue tem labels que facilitam a busca:

Label	Cor	Filtro no GitHub
<code>epic</code>	● Roxo	<code>label:epic</code>

user-story	 Azul	label:user-story
task	 Verde	label:task
ep01:sequencial		label:ep01:sequencial
ep02:cuda		label:ep02:cuda
ep03:openmp		label:ep03:openmp
ep04:benchmark		label:ep04:benchmark
ep05:docs		label:ep05:docs
priority:high		label:"priority:high"
priority:medium		label:"priority:medium"

Como cada Issue está estruturada

Todas as issues foram escritas com **alto nível de detalhamento** para que qualquer pessoa (ou IA) consiga entender e executar sem precisar de contexto adicional.

Épicos contêm:

- Descrição do objetivo macro (Como/Quero/Para)
- Escopo (incluído / fora de escopo)
- Contexto técnico e referências ao baseline
- Lista de User Stories vinculadas como sub-issues
- Dependências entre épicos
- Resultado esperado

User Stories contêm:

- Formato **Como/Quero/Para**
- Escopo detalhado
- Contexto técnico (referências ao baseline, linhas de código, decisões)
- Cenário / Fluxo passo a passo
- Checklist de sub-itens
- Orientações de implementação
- Lista de Tasks vinculadas como sub-issues
- Resultado esperado

Tasks contêm:

- Descrição do que precisa ser feito
- Arquivo(s) exato(s) a criar/modificar
- Exemplos de código e assinaturas de função
- Checklist de implementação
- Critérios de aceitação (o que é "pronto")
- Link para a User Story pai

Como tudo está linkado

A hierarquia é:

ÉPICO → contém User Stories → contém Tasks

Dentro de cada Issue vocês vão ver:

- No **Épico**: sub-issues com as USs (barra de progresso automática)
- Na **User Story**: sub-issues com as Tasks + referência ao Épico pai
- Na **Task**: referência à US pai no final da descrição

Conforme formos fechando Tasks/USs, as barras de progresso vão atualizando.

Estrutura do repositório

```
aco-selection-parallel/  
├── data/  
│   ├── baseline/      ← 9 datasets do paper original (separador ;)  
│   └── cdc/           ← CDC Diabetes, 253k instâncias (separador ;)  
├── src/  
│   ├── sequential/    ← versão C++ sequencial (a desenvolver)  
│   ├── cuda/          ← versão CUDA (a desenvolver)  
│   └── openmp/         ← versão OpenMP (a desenvolver)  
├── scripts/           ← download, benchmark, profiling, gráficos  
├── results/           ← saídas experimentais  
├── docs/              ← artigo + apresentação  
└── .refs/             ← materiais internos (NÃO vai pro git)
```

O que é o `.refs/`

Pasta **local** (ignorada no git) com materiais de referência:

- `proposta_inicial.md` — proposta entregue ao professor
- `prompt_trabalho.md` — contexto completo do projeto (usar como referência)
- `artigo-cad-colonia.pdf` — paper do baseline
- `paper_dataset.pdf` — paper do dataset CDC Diabetes
- `aco-cpp-main/` — código-fonte original do baseline

O dataset que estamos usando

CDC Diabetes Health Indicators — 253.680 instâncias, 21 features numéricas, target binário, sem missing values. É ~51× maior que o maior dataset do baseline. Já está baixado em `data/cdc/cdc_diabetes.csv`.

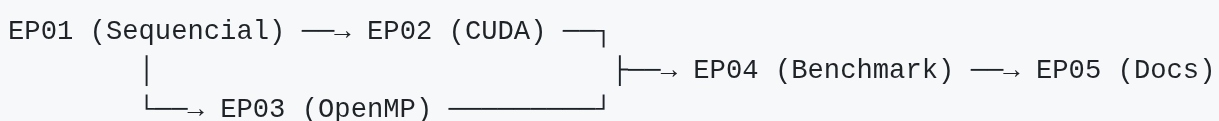
É nesse dataset que o CUDA vai mostrar vantagem real sobre o sequencial.

Como vamos trabalhar

1. **Pegar uma Task** → Vá na aba Issues, filtre por `label:task`, escolha uma que não tenha ninguém atribuído
2. **Se atribuir** → Clique em "Assignees" e se adicione
3. **Ler a descrição** → A Task tem tudo: arquivo, função, critérios. Leia tudo antes de começar
4. **Criar branch** → `git checkout -b feat/T01-makefile`
5. **Desenvolver** → Siga o detalhamento da Task
6. **Comitar** → Use Conventional Commits: `feat:`, `fix:`, `chore:`, `test:`, `docs:`
7. **Abrir PR** → Referencie a Issue no PR: `closes #N`
8. **Review + Merge** → Após aprovação, merge na main

Ordem de execução importa!

Os Épicos têm dependências entre si. Respeitem a ordem:



⚠ **Não comecem Tasks do EP02/EP03 antes de terminar o EP01.** ⚠ **Não comecem EP04 antes de ter pelo menos EP01 + EP02 prontos.**

Links úteis

Recurso	Link
 Project Board	https://github.com/users/phael-exe/projects/3
 Todas as Issues	https://github.com/phael-exe/aco-selection-parallel/issues
 Só Épicos	https://github.com/phael-exe/aco-selection-parallel/labels/epic
 Só User Stories	https://github.com/phael-exe/aco-selection-parallel/labels/user-story
 Só Tasks	https://github.com/phael-exe/aco-selection-parallel/labels/task
 Prioridade Alta	https://github.com/phael-exe/aco-selection-parallel/labels/priority%3Ahigh

Qualquer dúvida é só chamar! 🚀